Error Handling Helpline – Sprint 3

This document shows the development and implementation of Sprint 3 for the Error Handling Helpline app. The aim of this sprint was to deliver a working prototype using Node.js, Express, MySQL and Pug that demonstrates the core functionality and dynamic data integration between the database and user interface.

GitHub:

https://github.com/WILR96/ErrorHandlingHelpline

Task board:

https://github.com/users/WILR96/projects/1

Group Members

Reece Willoughby

Alexi Colley

Nawaraj Budhathoki

Implemented Features

During Sprint 3, these features were implemented using Node.js, Express, MySQL and Pug.

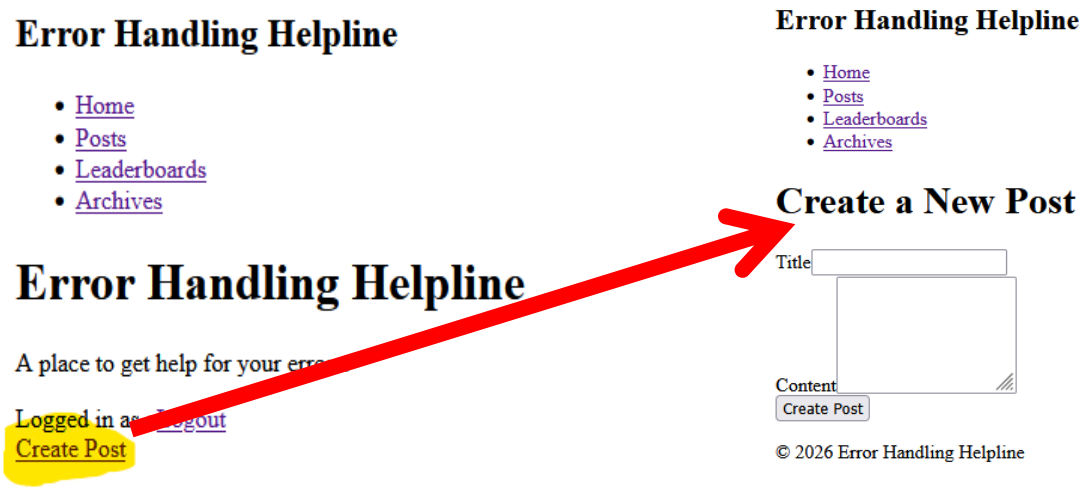
Create a Help Post:

As a user, I want to create a help request, so that I can receive assistance with a coding problem.

Implementation:

Authenticated users can create a new post by submitting a title and description through a form. The data is stored in the MySQL database and immediately displayed on the main posts feed.

Screenshots:



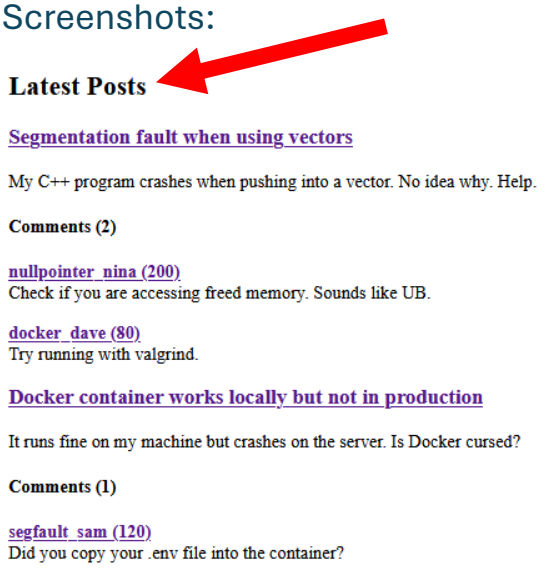
View Posts (Listing Page)

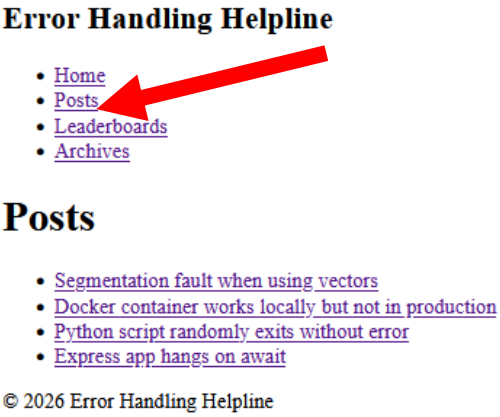
As a user, I want to browse help posts, so that I can find problems I can assist with.

Implementation:

All posts are retrieved from the database using SQL queries and displayed on the main page (/). Posts are ordered by most recent to get the most recent help posts. We use a pug template to dynamically render the page with new posts.

Screenshots:





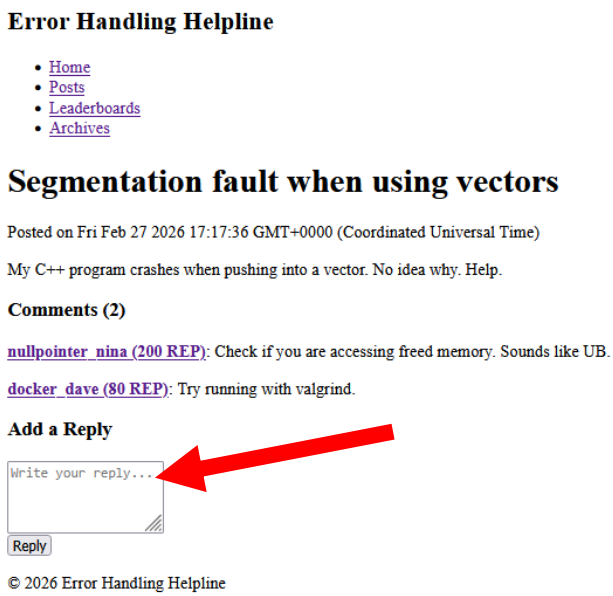
Answer a Post

As a user, I want to respond to a help request, so that I can assist another student.

Implementation:

Logged in users can submit responses to posts. Comments are stored in the database and displayed below the associated post. Each comment shows the author and their current reputation.

Screenshots:



View Other Users' Reputation

As a user, I want to see reputation next to usernames in posts, so that I can identify experienced contributors.

Implementation:

User reputation is retrieved from the database and displayed alongside usernames in comments. This is achieved using SQL joins between the "responses" and "users" tables

Screenshots:

Comments (2)

nullpointer_nina (200 REP): Check if you are accessing freed memory. Sounds like UB.

docker_dave (80 REP): Try running with valgrind.

User List

As a user, I want to view all users so that I can explore the community.

Implementation:

Users are retrieved from the database and displayed using a Pug template. The pug is setup so that the usernames are hyperlinked to their User Profile page.

Screenshot:

Error Handling Helpline

- Home
- Posts
- Leaderboards
- Archives

Leaderboard

- segfault_sam
- docker_dave
- nullpointer_nina
- panic_paul
- abc
- a

© 2026 Error Handling Helpline

User Profile Page (Basic)

As a user, I want to view a user's profile so that I can see their activity and reputation.

Implementation:

User data is retrieved using SQL queries and displayed dynamically.

Screenshot:

Post Categories

As a user, I want to filter posts by category so that I can quickly find help requests relevant to my area of interest.

Implementation:

A category dropdown has been added to the posts listing page. Categories are retrieved from the database and displayed using a Pug template. When a category is selected the application filters posts on the selected category using SQL filtering.

Screenshot:

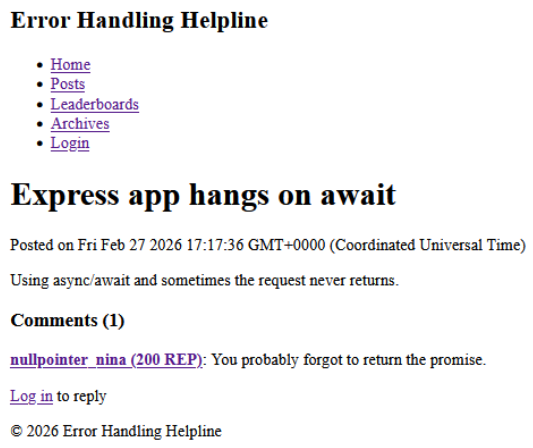
View Post (Details Page)

As a user, I want to view a single post in detail so that I can read the full help request and see all responses.

Implementation:

A single post is retrieved from the MySQL database using its unique ID and displayed on a dedicated page. All associated responses are also retrieved using SQL queries and shown beneath the post.

Screenshot:



Error Handling Helpline

- Home
- Posts
- Leaderboards
- Archives
- Login

Express app hangs on await

Posted on Fri Feb 27 2026 17:17:36 GMT+0000 (Coordinated Universal Time)

Using async/await and sometimes the request never returns.

Comments (1)

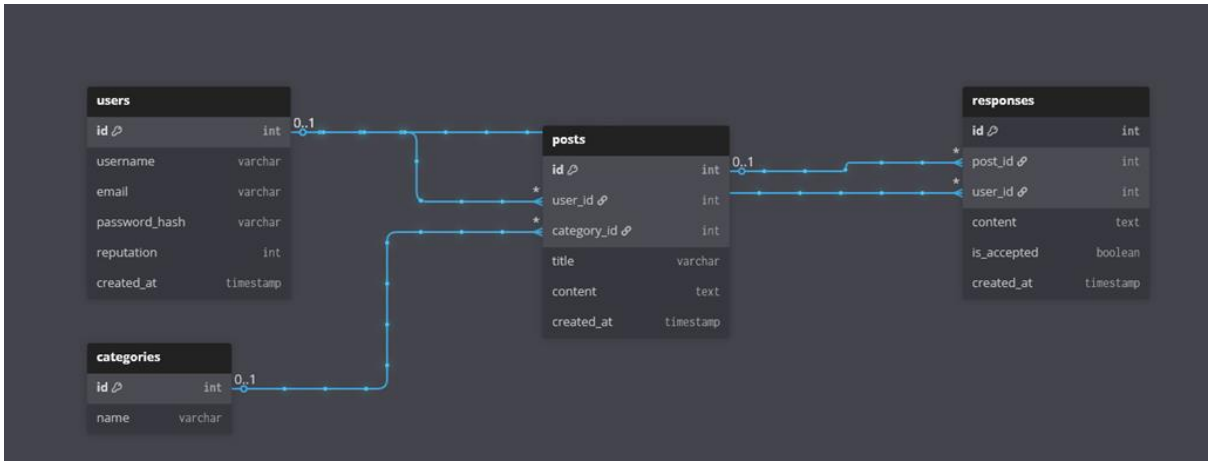
nullpointer_nina (200 REP): You probably forgot to return the promise.

Log in to reply

© 2026 Error Handling Helpline

Database Design

The application uses a relational database with the following structure:



Users:

The users table stores account information including username, email, password hash and reputation. Each user can create multiple posts and responses and the created_at timestamp records when their account was created. User data is retrieved from the database and displayed in the front-end using Pug templates.

Categories:

The categories table stores the names of post categories and is used to group posts for filtering and future functionality. Each category has a unique ID and name and is auto incremented so that more categories can be added in the future.

Posts:

The posts table stores help requests submitted by users. Each post is linked to a user by the user_id and associated with a category via category_id. Posts include a title, content, number of upvotes and a created_at timestamp. This allows the application to display posts in listing and detail pages.

Responses:

The responses table stores comments on posts. Each response is linked to both the post it belongs to (post_id) and the user who created it (user_id). Responses include content, upvotes, a boolean is_accepted field, and a timestamp. User reputation is retrieved from the users table and displayed alongside each response using SQL joins.

Database Relationships:

Relationships are enforced using foreign keys:

posts.user_id --> users.id

posts.category_id --> categories.id

responses.post_id --> posts.id

responses.user_id --> users.id

This design supports creating and viewing posts, responding to help requests, displaying user reputation and organizing posts by category.

Task Breakdown

Feature / Task	Description	Assigned Developer(s)	Status
Users List Page	Retrieve and display all users from database	Reece, Nawaraj	Completed
User Profile Page	Display individual user details and activity	Nawaraj	In Progress
Posts Listing Page	Display all posts ordered by date	Reece	Completed
Post Detail Page	Display single post with comments	Reece	Completed
Create Post Feature	Allow logged-in users to create posts	Reece	Completed
Comment System	Allow users to reply to posts	Reece	Completed
Authentication	Login and signup functionality with sessions	Reece	Completed
Database Design	Create relational schema and tables	Alexi (Finn)	Completed
Wireframes Update	Improve UI design based on user stories	Alexi (Finn)	In Progress
CSS Styling	Apply visual design to application	Nawaraj	Ready
Categories/Tags	Add filtering and categorisation	Reece	Completed
Sprint 3 Documentation	Prepare PDF submission	Reece	Completed

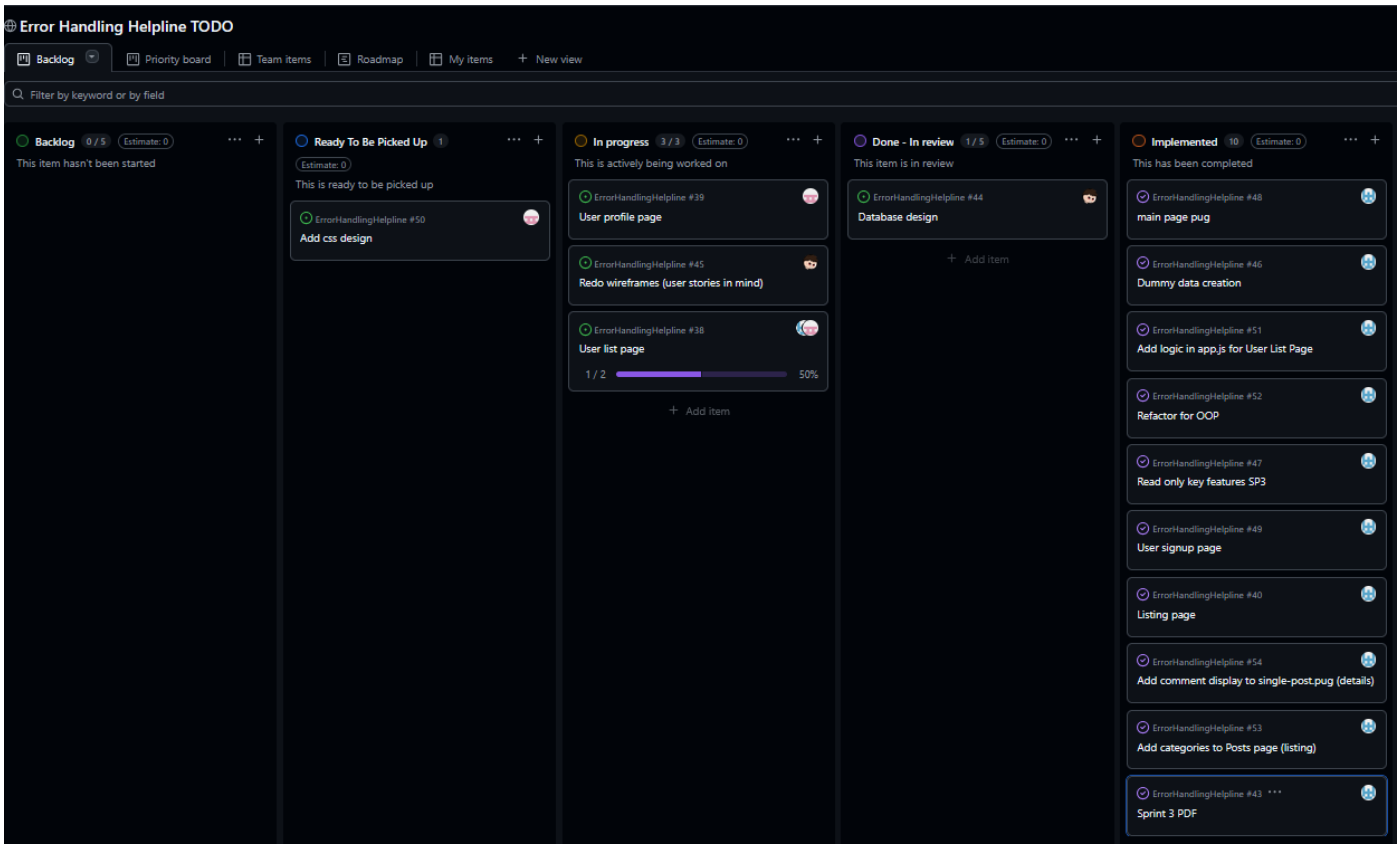
Task allocation was managed using a Kanban board within GitHub Projects. Tasks were distributed among team members and tracked across different stages including backlog, in progress, review and completed.

Core backend functionality and database integration were prioritised during this sprint to meet the minimum requirements. Remaining tasks focus on UI improvements and additional features, which will be completed in later sprints.

Evidence of Contribution

Kanban Board

The Kanban board was used to manage tasks, track progress, and assign responsibilities throughout the sprint.



Meeting Records

Date and Time	10/03/2026
Project Name	Error Handling Helpline

Meeting Goal	Review lecturer guidance for Sprint 3. Plan implementation of core features and tasks. Assign responsibilities for database, front-end templates, and styling.
Facilitator	Reece Willoughby
Note taker	Reece Willoughby
Attendees	**Reece Willoughby** **Alexi Colley** **Nawaraj Budhathoki**
Roundtable Updates (each group member to contribute)	**Reece:** Discussed instructions to focus on core features, including users, posts, responses, categories and upvotes. Stressed on the point of keeping Sprint 3 achievable by minimising optional features. **Alexi:** Suggested focusing on the database side.. refining the schema, ensuring foreign keys correctly support posts, responses, categories and votes. **Nawaraj:** Agreed to work on front-end implementation, designing a Pug template and CSS styling to match wireframes and user stories.
Discussion points	**Prioritise features for Sprint 3: create/view posts, view responses, categories, and upvotes.** **Database design should support all core functionality.**

	Generate dummy SQL data for testing. **Front-end work (Pug templates and CSS) to display users, posts and responses clearly.** **Optional features: messaging, awards, leaderboards, file uploads in SP4.**
Actions (list tasks and assign a group member)	Alexi - Align wireframes to updated user stories Alexi - Create UML for DB Nawaraj - Pug templates & CSS styling Alexi - Database design & UML refinement Reece - Dummy SQL data generation Reece - Core read-only features implementation Reece - Update Kanban board with tasks